

Boba Browser Extension – Complete Implementation Plan

Project: BobaLink – Browser Extension for Torrent Discovery & Download

Version: 1.0.0

Date: 2026-06-06

Executive Summary

BobaLink is a cross-browser extension (Chrome, Firefox, Opera, Yandex, Chromium) that detects torrent files and magnet links on web pages and in tab groups, then sends them to the Boba Project’s qBitTorrent web dashboard for downloading. Built on Manifest V3 with TypeScript, WXT build system, and comprehensive testing.

Phase 1: Foundation (Week 1)

1.1 Project Setup

Task	Sub-task	Description	Deliverable
1.1.1	Initialize repository	Git init, LICENSE (Apache 2.0), README	repo structure
1.1.2	Setup WXT build system	Install wxt, configure wxt.config.ts for 5 browsers	build config
1.1.3	TypeScript configuration	tsconfig.json with strict mode, @types/chrome	TS config
1.1.4	Linting & formatting	ESLint 9 flat config, Prettier, Husky pre-commit	quality config
1.1.5	Directory structure	src/, src/background/, src/content/, src/popup/, src/options/, src/shared/, src/types/, public/, tests/	folders

1.2 Manifest V3 Configuration

Task	Sub-task	Description	Deliverable
1.2.1	Base manifest.json	name, version, description, permissions, host_permissions	manifest
1.2.2	Chrome manifest		chrome manifest

Task	Sub-task	Description	Deliverable
1.2.3	Firefox manifest	MV3 with service worker, specific host patterns browser_specific_settings, gecko ID	firefox manifest
1.2.4	Opera manifest	sidebar action, minimum_opera_version	opera manifest
1.2.5	Yandex manifest	Chromium-compatible, tune:// URL handling	yandex manifest
1.2.6	Permission model	storage, alarms, notifications, activeTab, host_permissions	permissions

1.3 Shared Infrastructure

Task	Sub-task	Description	Deliverable
1.3.1	Type definitions	src/types/torrent.ts, config.ts, api.ts	type system
1.3.2	Logger utility	src/shared/logger.ts â€” structured logging with levels	logger
1.3.3	Constants	src/shared/constants.ts â€” URLs, ports, regex patterns	constants
1.3.4	Error types	src/shared/errors.ts â€” custom error classes	error handling
1.3.5	Event bus	src/shared/events.ts â€” typed event emitter for inter- module comm	event system

Phase 2: Core Engine (Week 2)

2.1 Magnet Link Detection & Parsing

Task	Sub-task	Description	Deliverable
2.1.1	Magnet regex engine	Fast detection regex + validation regex for BTIH	parser core
2.1.2	URI decoder	decodeURIComponent for dn, tr, xl parameters	decoder
2.1.3	BTIH validator	40-char hex validation, base32 support	validator
2.1.4	Parameter extractor	Extract xt, dn, tr, xl, ws, xs, as, kt, mt	extractor
2.1.5	Normalizer	Convert any magnet format to canonical {infoHash, name, trackers}	normalizer
2.1.6	Unit tests	50+ test cases covering all magnet formats	test suite

2.2 Torrent File Parsing

Task	Sub-task	Description	Deliverable
2.2.1	Bencode decoder	Zero-dependency Uint8Array-based decoder	decoder
2.2.2	Torrent parser class	Parse .torrent files, extract info dict, files	parser class
2.2.3	Infohash computer	SHA-1 of bencoded info dict via Web Crypto API	hasher
2.2.4	Magnet generator	Create magnet URI from parsed torrent data	generator
2.2.5	Validator	Check required fields (announce, info, piece length)	validator
2.2.6	Unit tests	Test with real .torrent files, edge cases	test suite

2.3 DOM Scanner Engine

Task	Sub-task	Description	Deliverable
2.3.1	Base scanner class	Abstract scanner with EventTarget interface	base class
2.3.2	Anchor link scanner	querySelectorAll(â€~a[href^=â€œmagnet:â€¢], a[href\$=â€œ.torrentâ€¢]â€™)	link scanner
2.3.3	Text node scanner	TreeWalker-based text scanning for inline magnets	text scanner
2.3.4	Site-specific selectors	Top 20 torrent site CSS selector database	site DB
2.3.5	MutationObserver setup	Debounce observer (500ms) for dynamic content	observer
2.3.6	Shadow DOM support	Recursive shadow root traversal	shadow scanner
2.3.7	Deduplication engine	Map by infoHash to prevent duplicates	deduper
2.3.8	Performance budget	requestAnimationFrame yielding, 16ms/frame limit	performance

Phase 3: Browser Extension Shell (Week 2-3)

3.1 Content Script

Task	Sub-task	Description	Deliverable
3.1.1	Entry point	src/content/index.ts â€” initialization	entry
3.1.2	Page scanner orchestrator	Initialize all scanners, collect results	orchestrator
3.1.3	Highlight/overlay UI	Visual indicators on detected torrents/magnets	overlay

Task	Sub-task	Description	Deliverable
3.1.4	Message handler	Listen for SCAN_REQUEST, return results to background	messaging
3.1.5	Auto-scan on load	document_idle injection, initial scan + observer	auto-scan
3.1.6	Manual scan trigger	Respond to background script scan command	manual scan

3.2 Background Service Worker

Task	Sub-task	Description	Deliverable
3.2.1	Entry point	src/background/index.ts â€” SW lifecycle management	entry
3.2.2	Message router	Route messages between content/popup/options	router
3.2.3	Keep-alive strategy	chrome.alarms for SW persistence	keep-alive
3.2.4	Context menu handler	â€”Send to Bobaâ€”, â€”Scan Pageâ€”, â€”Open Dashboardâ€”	context menu
3.2.5	Keyboard shortcuts	Ctrl+Shift+B (send), Ctrl+Shift+S (scan), etc.	shortcuts
3.2.6	Health check loop	Periodic Boba server ping via chrome.alarms	health
3.2.7	Badge updater	Update badge text/color based on status	badge
3.2.8	Notification handler	Show download start/complete/error notifications	notifications

3.3 Popup UI

Task	Sub-task	Description	Deliverable
3.3.1	HTML structure	src/popup/index.html â€” clean layout	HTML
3.3.2	CSS styling	src/popup/style.css â€” theme-aware, responsive	CSS
3.3.3	JavaScript logic	src/popup/popup.ts â€” torrent list, actions, state	JS
3.3.4	Connection status	Show Boba server connection state	status indicator
3.3.5	Detected torrents list	Scrollable list with checkboxes	torrent list
3.3.6	Send action	â€”Send Selected to Bobaâ€” button with progress	send flow
3.3.7	Batch operations	Select All, Deselect All, Invert Selection	batch actions
3.3.8	Quick settings		settings

Task	Sub-task	Description	Deliverable
		Toggle auto-scan, notification preferences	

3.4 Options Page

Task	Sub-task	Description	Deliverable
3.4.1	HTML structure	src/options/index.html â€” full settings page	HTML
3.4.2	CSS styling	src/options/style.css â€” clean form layout	CSS
3.4.3	JavaScript logic	src/options/options.ts â€” save/load settings	JS
3.4.4	Boba server config	URL, port, auth method, credentials	server config
3.4.5	Connection test	â€œTest Connectionâ€” button with visual feedback	test button
3.4.6	Auto-discovery	â€œAuto-Discover Bobaâ€” button scanning ports	discovery
3.4.7	Download preferences	Default category, save path, start paused	preferences
3.4.8	Security settings	Credential encryption toggle, HTTPS enforcement	security

Phase 4: API Integration (Week 3)

4.1 Boba API Client

Task	Sub-task	Description	Deliverable
4.1.1	Client class	src/shared/api/client.ts â€” BobaAPIClient	client class
4.1.2	Auth handlers	Cookie-based, API Key, Basic Auth support	auth
4.1.3	Health check	/app/version, /api/v2/app/ version endpoints	health
4.1.4	Auto-discovery	Scan localhost ports 7187, 7189, 8080	discovery
4.1.5	Error handling	Retry with exponential backoff, classify errors	error handler
4.1.6	Rate limiting	Token bucket limiter for API calls	rate limiter

4.2 qBitTorrent Direct Integration

Task	Sub-task	Description	Deliverable
4.2.1	Auth flow	Login, SID cookie management, CSRF handling	auth

Task	Sub-task	Description	Deliverable
4.2.2	Add torrents	POST /api/v2/torrents/add with urls/files	add torrent
4.2.3	Add magnets	Pass magnet: URIs via urls parameter	add magnet
4.2.4	Upload files	multipart/form-data for .torrent file upload	file upload
4.2.5	Monitor downloads	/torrents/info with hash filtering	monitor
4.2.6	Sync polling	/sync/maindata with incremental RID	sync

4.3 Offline Queue System

Task	Sub-task	Description	Deliverable
4.3.1	Queue class	src/shared/queue.ts "OfflineQueue"	queue
4.3.2	Persistence	chrome.storage.local for queued items	storage
4.3.3	Retry logic	Exponential backoff with jitter	retry
4.3.4	Process loop	chrome.alarms-based queue processor	processor
4.3.5	Conflict resolution	Skip if already added (check by infohash)	dedup

Phase 5: Tab Groups Integration (Week 3)

5.1 Tab Group Enumeration

Task	Sub-task	Description	Deliverable
5.1.1	Group query	chrome.tabGroups.query({}) for all groups	query
5.1.2	Tab extraction	chrome.tabs.query({groupId}) for URLs	extract
5.1.3	URL collection	Aggregate all tab URLs per group	collect
5.1.4	Batch scanning	Run content script scanner on each tab URL	scan

5.2 Context Menu Integration

Task	Sub-task	Description	Deliverable
5.2.1	Tab context menu	“Send Group to Boba” on right-click tab	menu item
5.2.2	Group submenu	List groups with torrent count per group	submenu

Task	Sub-task	Description	Deliverable
5.2.3	Action handler	Scan all tabs in group, collect, batch send	handler
5.2.4	Progress notification	Show “Scanning X tabs in group”	progress

Phase 6: UI/UX Polish (Week 4)

6.1 Icons & Theming

Task	Sub-task	Description	Deliverable
6.1.1	Icon set	16x16, 32x32, 48x48, 128x128 PNGs + SVG source	icons
6.1.2	Dynamic badges	Color-coded badges (green=active, blue=done, red=error)	badge states
6.1.3	Theme detection	prefers-color-scheme for dark/light mode	theme
6.1.4	Animation	Smooth transitions, loading spinners	polish

6.2 Internationalization

Task	Sub-task	Description	Deliverable
6.2.1	i18n setup	_locales/en/messages.json structure	i18n
6.2.2	English strings	All UI strings in en/messages.json	en strings
6.2.3	i18n helper	chrome.i18n.getMessage() wrapper	helper

6.3 Accessibility

Task	Sub-task	Description	Deliverable
6.3.1	ARIA labels	All interactive elements have aria-label	labels
6.3.2	Keyboard nav	Tab order, Enter/Space activation	keyboard
6.3.3	Focus management	Visible focus indicators, focus trapping	focus
6.3.4	Screen reader	Live regions for status updates	ally

Phase 7: Testing (Week 4-5)

7.1 Unit Tests

Task	Sub-task	Description	Deliverable
7.1.1	Jest configuration	jest.config.ts, ts-jest, jsdom environment	config
7.1.2	Mock APIs	chrome.* API mocks (storage, tabs, runtime, alarms)	mocks
7.1.3	Magnet parser tests	50+ test cases, edge cases, invalid inputs	tests
7.1.4	Torrent parser tests	Real torrent fixtures, bencode tests	tests
7.1.5	DOM scanner tests	jsdom-based DOM manipulation tests	tests
7.1.6	API client tests	Mock fetch, auth flow, error handling	tests
7.1.7	Queue tests	Persistence, retry, deduplication	tests
7.1.8	Coverage target	80%+ line coverage	coverage

7.2 Integration Tests

Task	Sub-task	Description	Deliverable
7.2.1	API integration	Test against real Boba docker instance	tests
7.2.2	Auth flow	Cookie auth, API key auth, error cases	tests
7.2.3	End-to-end flow	Detect â†’ Parse â†’ Send â†’ Verify download	tests

7.3 E2E Tests

Task	Sub-task	Description	Deliverable
7.3.1	Playwright config	playwright.config.ts with extension loading	config
7.3.2	Extension fixture	Load extension in Chromium/Chrome	fixture
7.3.3	Popup tests	Open popup, verify UI, click actions	tests
7.3.4	Content script tests	Visit torrent page, verify detection	tests
7.3.5	Options tests	Configure server, test connection	tests

Phase 8: Build & Distribution (Week 5)

8.1 Build Pipeline

Task	Sub-task	Description	Deliverable
8.1.1	WXT build	wxt build for production	build
8.1.2	Cross-browser builds	Chrome, Firefox, Opera, Yandex packages	packages
8.1.3	CI/CD workflow	.github/workflows/ci.yml â€” lint, test, build	CI
8.1.4	Release workflow	.github/workflows/release.yml â€” publish to stores	CD

8.2 Store Submission

Task	Sub-task	Description	Deliverable
8.2.1	Chrome Web Store	Screenshots, description, privacy policy, \$5 fee	submission
8.2.2	Firefox AMO	web-ext sign, listing page, review responses	submission
8.2.3	Opera Addons	Opera-specific build, sidebar screenshot	submission
8.2.4	Yandex	Chromium-compatible package, description	submission

Deliverables Checklist

Source Code

- ☐ src/ directory with all TypeScript source
- ☐ wxt.config.ts build configuration
- ☐ manifest configuration per browser
- ☐ package.json with all dependencies
- ☐ tsconfig.json, jest.config.ts, playwright.config.ts
- ☐ .github/workflows/ci.yml, release.yml

Tests

- ☐ Unit tests (Jest) â€” 80%+ coverage
- ☐ E2E tests (Playwright)

- ☐ Test fixtures (sample .torrent files, HTML pages)
- ☐ Mock implementations for browser APIs

Documentation

- ☐ Technical Specification (architecture, components, APIs)
- ☐ API Reference (all internal and external APIs)
- ☐ User Guide (installation, configuration, usage)
- ☐ Developer Guide (setup, build, test, contribute)
- ☐ Installation Guide (per-browser instructions)

Diagrams

- ☐ System architecture (Mermaid + SVG + PNG)
- ☐ Data flow diagrams (Mermaid + SVG + draw.io)
- ☐ Sequence diagrams (Mermaid + UML)
- ☐ ER diagram (database schema)
- ☐ Component diagram
- ☐ Deployment diagram

Multi-Format Output

- ☐ All docs in Markdown
- ☐ All docs in PDF (enterprise styled)
- ☐ All docs in DOCX (enterprise styled)
- ☐ All docs in HTML (clean styled)
- ☐ All diagrams in Mermaid, SVG, PNG, draw.io, UML

SQL Schema Definitions

Extension Local Storage (`chrome.storage.local`)

```
-- Configuration Table
CREATE TABLE IF NOT EXISTS extension_config (
```

```

        key TEXT PRIMARY KEY,
        value TEXT NOT NULL,
        updated_at INTEGER NOT NULL
    );

-- Discovered Torrents Table (session)
CREATE TABLE IF NOT EXISTS discovered_torrents (
    id TEXT PRIMARY KEY, -- infoHash
    page_url TEXT NOT NULL,
    magnet_uri TEXT,
    torrent_url TEXT,
    name TEXT,
    trackers TEXT, -- JSON array
    size_bytes INTEGER,
    source_type TEXT CHECK(source_type IN ('magnet-link', 'torrent-file',
    discovered_at INTEGER NOT NULL,
    sent_to_boba INTEGER DEFAULT 0,
    boba_status TEXT CHECK(boba_status IN ('pending', 'queued', 'added', '
);

-- Download Queue (offline support)
CREATE TABLE IF NOT EXISTS download_queue (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    info_hash TEXT NOT NULL,
    magnet_uri TEXT,
    torrent_data BLOB, -- base64 encoded .torrent file
    name TEXT,
    category TEXT,
    save_path TEXT,
    added_at INTEGER NOT NULL,
    retry_count INTEGER DEFAULT 0,
    last_error TEXT,
    status TEXT CHECK(status IN ('pending', 'retrying', 'failed', 'complet
);

-- Boba Server Configuration
CREATE TABLE IF NOT EXISTS server_config (
    id INTEGER PRIMARY KEY CHECK(id = 1),
    base_url TEXT NOT NULL,
    auth_method TEXT CHECK(auth_method IN ('none', 'cookie', 'api_key', 'b
    api_key TEXT,
    username TEXT,
    password_encrypted TEXT,
    is_reachable INTEGER DEFAULT 0,
    last_check INTEGER,
    qbittorrent_version TEXT,
    boba_version TEXT
);

-- Send History
CREATE TABLE IF NOT EXISTS send_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    info_hash TEXT NOT NULL,
    name TEXT,

```

```
    sent_at INTEGER NOT NULL,  
    success INTEGER NOT NULL,  
    error_message TEXT,  
    server_url TEXT NOT NULL  
);
```

Boba API Endpoints (External “ Reference)

```
-- qBitTorrent torrents table (reference only)  
-- Actual storage is managed by qBittorrent internally  
-- Extension polls /api/v2/torrents/info to get this data  
  
-- Search results cache (optional, for offline viewing)  
CREATE TABLE IF NOT EXISTS search_cache (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    query TEXT NOT NULL,  
    category TEXT,  
    results_json TEXT NOT NULL, -- JSON array of MergedResult  
    cached_at INTEGER NOT NULL,  
    expires_at INTEGER NOT NULL  
);
```

Gantt Chart (Estimated Timeline)

Week 1		Phase 1: Foundation + Phase 2: Core Engine
Week 2		Phase 2: Core Engine (cont) + Phase 3: Extension Shell
Week 3		Phase 3: Extension Shell (cont) + Phase 4: API Integration + Pha
Week 4		Phase 6: UI/UX Polish + Phase 7: Testing (start)
Week 5		Phase 7: Testing (cont) + Phase 8: Build & Distribution

Risk Assessment

Risk	Impact	Mitigation
qBitTorrent API changes	High	Version detection, adapter pattern
CORS on torrent sites	Medium	Service worker fetch bypasses CORS
MV3 service worker termination	Medium	chrome.alarms keep-alive strategy
Store review rejection	Medium	Follow guidelines, minimal permissions
Yandex Browser API differences	Low	Test on Yandex, Chromium-compatible code